

Industrial Internship Report on Content Management System for Blog

Prepared by
Srivani Koppu

Executive Summary

This report presents the work completed during my industrial internship in **Full Stack Web Development**. The project involved developing a **Content Management System (CMS) for a Blog** using Node.js, Express.js, EJS, MySQL, HTML, CSS, and JavaScript.

The system includes user authentication, blog management, a drag-and-drop page builder, media management, comments, contact management, search and filtering, user settings, password reset, and account deletion. All major features were implemented, integrated, tested, and deployed successfully.

The internship provided practical experience in full-stack development, database management, debugging, testing, deployment, and building a complete web application from development to production.

TABLE OF CONTENTS

Chapter Title	Page
1 Preface	4
2 Introduction	5
2.1 About UniConverge Technologies Pvt. Ltd.	5
2.2 About Upskill Campus	6
2.3 About The IoT Academy	6
2.4 Objectives of the Internship	6-7
2.5 Reference	7
2.6 Glossary	7-8
3 Problem Statement	9
4 Existing and Proposed Solution	10
4.1 Code Submission	10
4.2 Report Submission	10
5 Proposed Design / Model	11
5.1 High-Level Diagram	11
5.2 Low-Level Design	12
5.3 Interfaces	12
6 Performance Test	13
6.1 Test Plan / Test Cases	13-14

Chapter Title	Page
6.2 Test Procedure	14-15
6.3 Performance Outcome	15
7 My Learnings	16
8 Future Work Scope	17

1 Preface

This industrial internship provided me with an opportunity to gain practical experience in Full Stack Web Development by designing and developing a complete **Content Management System (CMS) for a Blog**.

During the internship, I worked on different stages of the project, including requirement analysis, database design, frontend development, backend implementation, feature integration, testing, debugging, and deployment. The application was developed using technologies such as Node.js, Express.js, EJS, MySQL, HTML, CSS, and JavaScript.

The project includes major features such as user registration and login, password recovery, profile and account management, blog creation and publishing, a drag-and-drop page builder, media management, comments and moderation, contact management, search and filtering, website settings, and public page rendering.

Throughout the internship, I faced and resolved various challenges related to routing, database operations, validation, authentication, file uploads, UI design, and deployment. This helped me improve my problem-solving and debugging skills and understand how different components of a full-stack application work together.

The internship also helped me understand the importance of database consistency, secure authentication, input validation, responsive design, testing, and production deployment. It provided valuable exposure to real-world software development practices and improved my confidence in developing complete web applications.

I sincerely thank Upskill Campus, The IoT Academy, UniConverge Technologies Pvt. Ltd., my mentors, faculty members, and everyone who supported and guided me throughout the internship. Their guidance and feedback played an important role in successfully completing the project.

Overall, this internship was a valuable learning experience that strengthened my technical knowledge and provided a strong foundation for my future career in **Full Stack Web Development**.

2 Introduction

The rapid growth of web-based content has increased the need for systems that allow users to create, organize, manage, and publish digital information efficiently. A Content Management System provides a structured environment through which users can manage website content without having to manually modify every part of a website.

The project developed during this internship is a **Content Management System for Blog and Website Management**. The system provides a centralized platform where authenticated users can manage blogs, create web pages, upload media, moderate comments, manage contact messages, configure website settings, and control their account.

The application follows a client-server architecture in which the frontend provides the user interface, the Express.js backend manages application logic and routing, and the MySQL database stores application data. EJS is used to dynamically generate web pages based on database information and user actions.

The system was developed incrementally. Individual modules were first implemented and then integrated with other modules. Functional testing was performed throughout development to identify and resolve database, routing, validation, authentication, and interface-related issues.

2.1 About UniConverge Technologies Pvt Ltd

According to the internship reference material, **UniConverge Technologies Pvt. Ltd. (UCT)** was established in 2013 and works in the Digital Transformation domain, with a focus on sustainability and return on investment. The organization leverages technologies including Internet of Things, Cyber Security, Cloud Computing, Machine Learning, communication technologies, Java Full Stack, Python, and frontend technologies to develop industrial solutions.

UCT also provides IoT-based solutions through its platforms and works in areas such as industrial monitoring, smart cities, agriculture technology, smart metering, and predictive maintenance. Its solutions involve the collection, processing, and analysis of data generated by devices and industrial systems.

The company also works with technologies such as MQTT, CoAP, HTTP, Modbus TCP, and OPC UA for device connectivity. Its solutions can support cloud and on-premises deployment environments.

The industrial exposure provided through the internship helped me understand how software technologies can be applied to practical business and engineering requirements.

2.2 About upskill Campus (USC)

Upskill Campus is a career development platform that focuses on skill development and provides learning and career-oriented support to students and professionals. According to the internship material, Upskill Campus works to provide learners with opportunities for self-paced learning along with additional support services such as internships, projects, interaction with industry experts, and career development services.

The internship program provided a structured environment in which participants could work on practical projects while improving their technical and professional skills. The program helped bridge the gap between academic learning and practical software development.

2.3 The IoT Academy

The **IoT Academy** is the EdTech division associated with UCT and provides technology-oriented learning and certification programs. According to the provided internship material, the academy conducts executive certification programs in collaboration with academic and technology institutions.

The internship provided an opportunity to gain practical exposure to software development practices and understand the importance of implementing solutions based on real-world requirements.

2.4 Objectives of this Internship program

The major objectives of the internship were:

- To gain practical experience in software development.
- To understand the complete development lifecycle of a web application.
- To develop a real-world full-stack application.
- To understand frontend and backend integration.
- To gain practical knowledge of database management.
- To implement authentication and authorization.
- To improve debugging and problem-solving skills.
- To understand CRUD operations.
- To learn file upload and media management.
- To implement dynamic web pages.

- To develop a drag-and-drop page builder.
- To understand application testing.
- To learn Git and GitHub for version control.
- To gain experience in application deployment.
- To understand production environment configuration.

2.5 Reference

- [1] Express.js Documentation
- [2] Node.js Documentation
- [3] MySQL Documentation
- [4] EJS Documentation
- [5] Multer Documentation
- [6] Nodemailer Documentation
- [7] CKEditor Documentation
- [8] Git and GitHub Documentation
- [9] Railway Deployment Documentation
- [10] HTML, CSS and JavaScript documentation and learning resources

2.6 Glossary

Term	Description
CMS	Content Management System
CRUD	Create, Read, Update and Delete
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
Node.js	JavaScript runtime environment

Express.js	Node.js web application framework
EJS	Embedded JavaScript Templates
MySQL	Relational Database Management System
SQL	Structured Query Language
API	Application Programming Interface
UI	User Interface
UX	User Experience
URL	Uniform Resource Locator
SMTP	Simple Mail Transfer Protocol
Git	Version Control System
GitHub	Code hosting and collaboration platform

3 Problem Statement

Traditional website development often requires users to have technical knowledge of HTML, CSS, JavaScript, database operations, and server-side programming. Managing different types of website content separately can also make the process complicated, particularly when users need to manage blogs, pages, images, comments, and contact messages from different interfaces.

The problem addressed by this project was to develop a centralized web-based Content Management System that could simplify website and blog management through a single application. The system needed to provide authentication and account management while also allowing users to create and manage website content.

The proposed application therefore provides a combination of content management and website-building capabilities. Users can create blogs, manage their pages through a drag-and-drop page builder, upload and manage media, handle comments, view contact messages, configure website settings, and publish content through a public website interface.

The system was designed with the objective of providing an organized and user-friendly environment while maintaining proper database relationships and access control.

4 Existing and Proposed solution

Existing Content Management Systems provide various facilities for managing websites and digital content. Common CMS platforms support features such as blog publishing, media management, user authentication, comments, and website customization. However, such systems can sometimes involve complex configurations or additional extensions when users require multiple functionalities within a single platform.

The proposed solution is a web-based Content Management System that integrates the major website management functions into one application. Instead of maintaining separate systems for blogs, pages, media, comments, contact messages, and account management, the developed application provides these capabilities through a centralized CMS dashboard.

The system includes user authentication with registration, login, logout, password change, and email-based password recovery. Users can manage their profile information and permanently delete their accounts. The blog module supports creating, editing, publishing, searching, filtering, pagination, featured images, categories, tags, and unique slugs.

A major feature of the proposed system is the **drag-and-drop page builder**, which allows users to create pages using components such as headings, paragraphs, buttons, images, dividers, and spacers. Created pages can be saved, loaded, edited, previewed, and published.

The system also includes media management, comment moderation, contact management, website settings, and public page rendering. These functions work together through the backend and MySQL database to provide an integrated CMS environment

4.1 Code submission (Github link)

<https://github.com/srivanif9/upskillcampus.git>

4.2 Report submission (Github link) :

<https://github.com/srivanif9/upskillcampus>

5 Proposed Design/ Model

The proposed system follows a structured web application architecture consisting of the presentation layer, application layer, and database layer. The presentation layer provides the user interface through EJS templates, HTML, CSS, and JavaScript. The application layer is implemented using Node.js and Express.js and handles routing, authentication, business logic, validation, file uploads, and communication with the database. The database layer uses MySQL for storing users, pages, blogs, media, comments, contacts, settings, and activity information.

When a user performs an operation through the interface, the request is sent to the appropriate Express.js route. Middleware is used where authentication or file handling is required. The corresponding controller processes the request and communicates with MySQL. The result is then returned to the user through an EJS page or a JSON response, depending on the operation.

The application uses session-based authentication to restrict access to protected CMS functionality. Passwords are stored using hashing rather than plain text. Password recovery uses a secure token with a limited validity period.

5.1 High Level Diagram (if applicable)

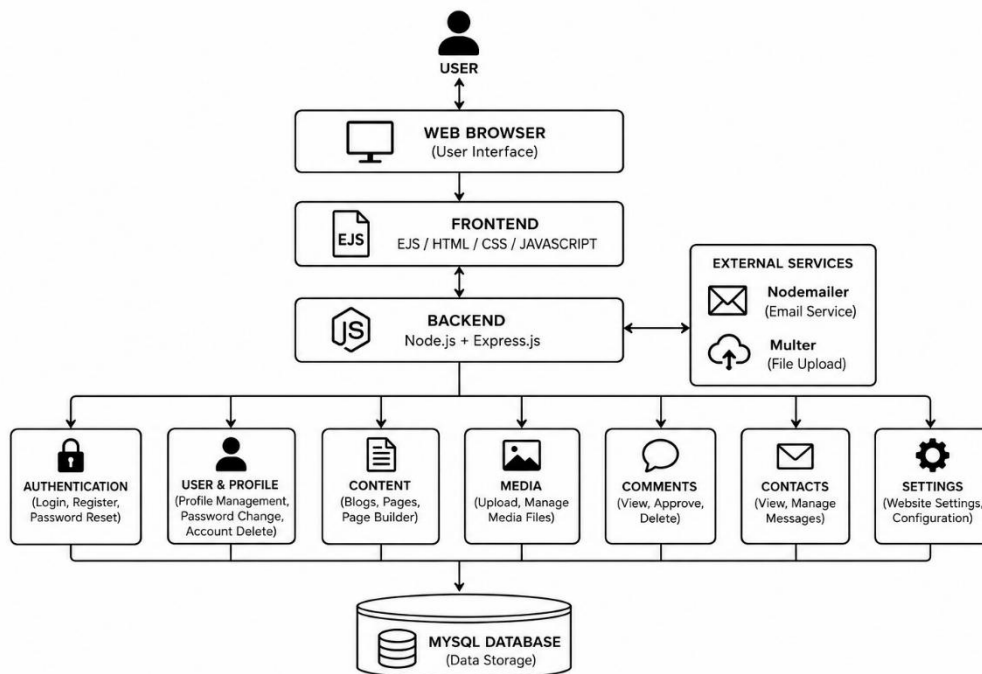


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

The low-level implementation is divided into routes, controllers, middleware, views, utility functions, and database configuration. Routes define the URLs and HTTP methods through which users interact with the application. Controllers contain the main application logic and perform database operations.

Authentication routes manage registration, login, logout, forgot-password requests, and password resetting. Profile routes manage profile viewing, editing, password changes, and account-related operations. Blog and page routes handle content creation and management. Media routes manage uploaded files, while comment and contact routes handle visitor interactions.

The MySQL database maintains relationships between users and their associated content. The `users` table stores authentication and account information. The `pages`, `blogs`, `media`, and `activity_logs` tables maintain content and user activity information. Foreign key relationships help maintain data consistency.

The page builder stores the page structure as JSON data in the database. This allows the application to reconstruct the page layout when the user loads or previews a page.

5.3 Interfaces (if applicable)

The application provides several interfaces for different CMS functions. The login and registration interfaces provide authentication and account creation. The dashboard acts as the central navigation area from which users can access CMS functions.

The blog management interface allows users to create, edit, search, filter, publish, and manage blog posts. The page builder provides a visual interface where components can be dragged onto a canvas and arranged to create a page. Saved layouts can later be loaded and edited.

The media management interface allows users to upload and manage images and other supported media. The comment management interface allows administrators or authorized users to review, approve, and delete comments. The contact management interface provides access to messages submitted through the public contact form.

The settings interface allows users to configure website information and manage account-related functions. The profile and authentication interfaces provide facilities for updating user information, changing passwords, recovering forgotten passwords, and permanently deleting accounts.

The public website interface displays published pages and blog content to visitors without requiring access to the CMS dashboard.

6 Performance Test

Testing was performed throughout the development process to verify the functionality, reliability, and integration of the CMS. Testing focused primarily on functional correctness, database consistency, routing, authentication, validation, file handling, and user interaction.

Each major module was tested individually before being integrated with the complete application. After integration, end-to-end testing was performed to verify that information could correctly flow between the frontend, backend, and database.

The testing process included both valid and invalid user inputs. Duplicate usernames, duplicate email addresses, duplicate page names, incorrect passwords, invalid reset links, missing form fields, incorrect password confirmation, and other invalid conditions were tested to verify that appropriate responses were generated.

6.1 Test Plan/ Test Cases

Test Case Functionality		Expected Result	Status
TC01	User Registration	New user should be registered successfully	Passed
TC02	Duplicate Registration	Duplicate username/email should be rejected	Passed
TC03	User Login	Valid credentials should allow login	Passed
TC04	Invalid Login	Incorrect credentials should be rejected	Passed
TC05	Logout	User session should be terminated	Passed
TC06	Forgot Password	Reset email should be sent to registered email	Passed
TC07	Reset Password	Valid reset token should allow password change	Passed
TC08	Expired Reset Link	Expired token should be rejected	Passed
TC09	Change Password	Correct current password should allow password change	Passed
TC10	Edit Profile	Profile details should be updated	Passed
TC11	Delete Account	Account should be permanently removed	Passed

Test Case Functionality		Expected Result	Status
TC12	Create Blog	New blog should be stored	Passed
TC13	Edit Blog	Existing blog should be updated	Passed
TC14	Publish Blog	Blog should become publicly accessible	Passed
TC15	Blog Search	Matching blogs should be displayed	Passed
TC16	Blog Filtering	Selected category/filter should work	Passed
TC17	Create Page	New page should be saved	Passed
TC18	Load Page	Saved page should load into builder	Passed
TC19	Preview Page	Page preview should be displayed	Passed
TC20	Publish Page	Page should become publicly accessible	Passed
TC21	Media Upload	File should be uploaded successfully	Passed
TC22	Comment Moderation	Comments should be approved/deleted correctly	Passed
TC23	Contact Form	Visitor message should be stored	Passed
TC24	Website Settings	Settings should be saved and displayed	Passed
TC25	Public Page	Published page should render correctly	Passed

6.2 Test Procedure

The testing procedure began with user authentication. Registration was tested using valid and duplicate account details to verify both successful registration and duplicate-record validation. Login was tested with valid and invalid credentials, followed by logout testing to verify session termination.

The password management functionality was tested using both registered and unregistered email addresses. Password reset emails were verified, and reset links were tested within and beyond their validity period. Password changes were also tested using correct and incorrect current passwords and mismatched confirmation passwords.

The blog module was tested by creating new blog posts and verifying their storage in the database. Editing, deletion, publishing, draft status, featured images, categories, tags, unique slugs, search, filtering, and pagination were also verified.

The page builder was tested by dragging components onto the canvas and saving the resulting layout. Previously saved pages were loaded and edited, and the preview and publish functions were tested. Public URLs were then accessed to verify that published pages were rendered correctly.

Media management was tested by uploading files, viewing uploaded media, selecting media for content, previewing files, and deleting media. The comment module was tested by submitting comments and verifying their moderation, approval, display, and deletion behavior.

The contact module and website settings were also tested. Finally, the complete application was tested after deployment to ensure that the major functions continued to work in the hosted environment.

6.3 Performance Outcome

The testing process confirmed that the major functional requirements of the CMS were successfully implemented. Authentication, profile management, password recovery, account deletion, blog management, page building, media management, comments, contacts, settings, search, filtering, preview, publishing, and public page rendering were successfully tested.

Several issues were identified during development and testing, particularly in routing, database operations, page-builder state management, duplicate validation, file handling, and deployment configuration. These issues were investigated and corrected before final deployment.

The final application was successfully deployed and the major modules were verified in the hosted environment. The testing results indicate that the application performs the intended CMS operations correctly under the tested conditions.

7 My learnings

This internship provided practical experience in developing a complete full-stack web application from the initial design stage to final deployment. I gained a better understanding of how frontend interfaces communicate with backend services and how backend applications interact with relational databases.

Through the project, I strengthened my knowledge of **Node.js and Express.js** for server-side development and routing. I learned how to organize an application using routes, controllers, middleware, views, and utility functions. Working with EJS helped me understand server-side rendering and dynamic generation of web pages.

My experience with **MySQL** improved significantly through the implementation of database tables, SQL queries, foreign key relationships, CRUD operations, and data validation. I learned the importance of maintaining relationships between users and their associated content and ensuring database consistency during deletion and update operations.

The authentication module provided practical understanding of session management, password hashing, login validation, password changes, and email-based password recovery. Implementing password reset functionality also helped me understand the importance of temporary tokens and token expiration.

The page builder was particularly useful for understanding frontend interaction and dynamic content generation. Implementing drag-and-drop components, saving layouts as JSON, loading existing layouts, previewing pages, and publishing pages helped improve my JavaScript and application-state management skills.

I also gained experience in file upload handling using Multer, rich-text content management, email integration using Nodemailer, frontend styling, debugging, and deployment. Testing the application helped me understand how individual modules can behave correctly on their own but still require additional testing after integration.

The internship also improved my ability to analyze errors and identify whether a problem was related to the frontend, backend, database, routing, configuration, or deployment environment. This practical debugging experience will be useful in future software development work.

Overall, the internship improved both my technical and problem-solving abilities and gave me greater confidence in developing complete web applications.

8 Future work scope

Although the developed CMS successfully implements the required functionality, several improvements can be considered for future versions. The page builder can be enhanced with additional components, more advanced styling controls, responsive layout options, reusable templates, and improved drag-and-drop behavior. These improvements would provide users with greater flexibility when designing websites.

The blog management system can be extended with additional features such as scheduled publishing, advanced content analytics, richer SEO controls, automatic metadata generation, and more detailed author management. A more advanced dashboard could provide graphical statistics about blog views, comments, visitors, and published content.

The authentication system can be further enhanced with features such as two-factor authentication, stronger account security controls, email verification, and improved session management. Password recovery can also be extended with additional security mechanisms and configurable email templates.

The media management module can be improved by adding image resizing, automatic optimization, file categorization, storage management, and cloud-based media storage. This would make the system more suitable for applications containing a large amount of media.

For production deployment, the application can be further improved by using a persistent session store instead of an in-memory session store, implementing stronger production security settings, configuring automated backups, adding monitoring and logging services, and improving database scalability.

The application can also be developed into a multi-user CMS where different users are assigned roles and permissions such as administrator, editor, author, and viewer. This would make the system more suitable for organizations where multiple people need to manage website content.

Future development could also include a REST API, mobile-friendly administrative interfaces, cloud storage integration, automated testing, CI/CD pipelines, and improved scalability. These enhancements would allow the CMS to evolve from an internship project into a more comprehensive production-oriented content management platform.

